

Mała sieć czujników do roślin całorocznych

BES

Autor: Mikołaj Drózdź

Data: 24 czerwca 2026

Spis treści

1	Wstęp	2
2	Podstawa sprzętowa i źródła projektu	2
3	Architektura oprogramowania	3
4	Cykl pracy — co godzinę	4
5	Format ramki telemetrycznej	5
5.1	Schemat ramki	5
5.2	Opis pól	5
5.3	CRC8-ATM	6
6	Odczyt czujników	7
6.1	Temperatura i ciśnienie — BMP280	7
6.2	Wilgotność gleby — pojemnościowe czujniki ADC	7
7	Parametry transmisji LoRa	8
7.1	Czas transmisji na powietrzu (ToA)	8
8	Wyjście terminala	9
8.1	Terminal węzła (urządzenie)	9
8.2	Terminal centrali (stacja bazowa)	10
9	Szacowany czas pracy na bateriach	11
9.1	Założenia sprzętowe	11
9.2	Obliczenie czasu pracy	11
9.3	Korekty praktyczne	11
9.4	Wymagania dla trybu Stop MCU z RTC	12

1 Wstęp

Projekt realizuje sieć bezprzewodowych węzłów czujnikowych przeznaczonych do monitorowania warunków dla roślin całorocznych (wilgotność gleby, temperatura i ciśnienie atmosferyczne). Węzły wysyłają zebrane dane do centralnej stacji bazowej za pomocą technologii LoRa (*Long Range*) na paśmie 868.1 MHz.

Niniejsza dokumentacja opisuje pojedynczy moduł, komponent odpowiedzialny za cykliczne odczytywanie czujników, pakowanie danych w ramkę o stałej strukturze oraz zlecenie transmisji przez radio.

2 Podstawa sprzętowa i źródła projektu

Projekt został zrealizowany na dedykowanych płytkach z nakładkami pochodzącymi z przedmiotu BEKO Sprzęt obejmuje:

- Płytką główną z MCU **STM32U545RETXQ** (STM32U5, Cortex-M33, ultra-low-power, 160 MHz).
- Nakładką z modułem **RFM95W** (SX1276, LoRa 868 MHz).
- Nakładką z czujnikami: **BMP280** (I2C) i dwoma pojemnościowymi czujnikami wilgotności gleby (ADC).

Kod źródłowy firmware opiera się na szablonie **master-rtos** z repozytorium autora (github.com/MikolajDrozdz), który dostarcza:

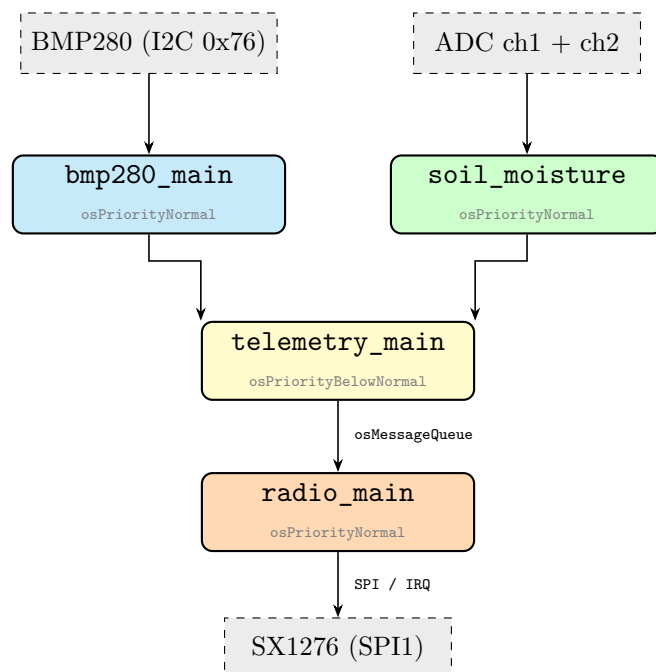
- szkielet FreeRTOS z warstwą CMSIS-OS2,
- sterownik I2C z retry i recovery (`app.c`),
- bibliotekę radiową `radio_lib` (backend LoRa/FSK/OOK),
- system modułów `*_main.c` dla każdego podsystemu.

Projekt telemetryczny BEKO rozszerza szablon o moduł czujnikowy (`telemetry_main.c`, `soil_moisture_main.c`) i optymalizację energetyczną radia (niniejsza dokumentacja).

3 Architektura oprogramowania

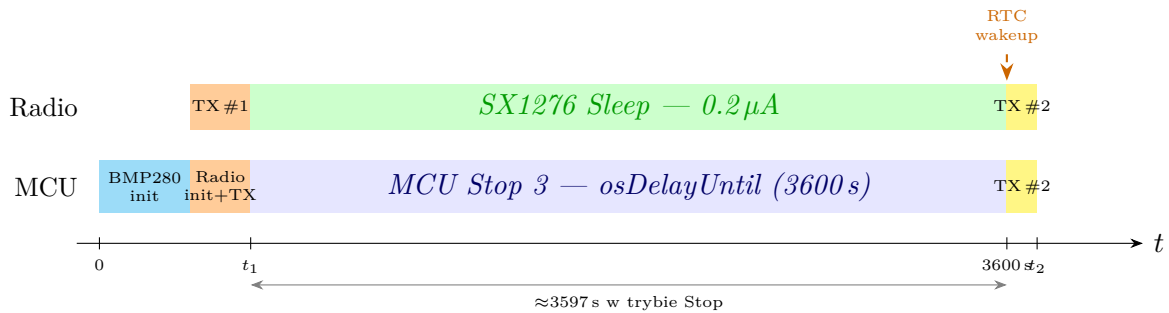
Firmware opiera się na FreeRTOS / CMSIS-OS2. Każdy podsystem działa jako osobne zadanie (task). Moduł telemetryczny współpracuje z trzema innymi modułami:

- `bmp280_main` — cykliczne pomiary temperatury i ciśnienia przez I2C (BMP280 pod adresem 0x76),
- `soil_moisture_main` — odczyt 2 pojemnościowych czujników wilgotności gleby przez 14-bitowy ADC (uśrednianie z 16 próbek),
- `radio_main` — kolejka TX i obsługa zdarzeń SX1276 przez SPI.



Rysunek 1: Graf zależności zadań FreeRTOS w module telemetrycznym

4 Cykl pracy — co godzinę



Rysunek 2: Diagram czasowy cyklu godzinnego — tryb energooszczędny (nie w skali)

Sekwencja działania modułu `telemetry_main_task_fn` (plik `telemetry_main.c`):

1. Oczekiwanie na pierwsze prawidłowe odczyty BMP280 (do 5 s, polling co 100 ms).
2. Wywołanie `telemetry_main_send_sample()` — pierwszy pomiar.
3. Pętla nieskończona: `osDelayUntil()` na 3600 s, po czym kolejny pomiar i transmisja.

Listing 1: Fragment `telemetry_main.c` — pętla główna

```

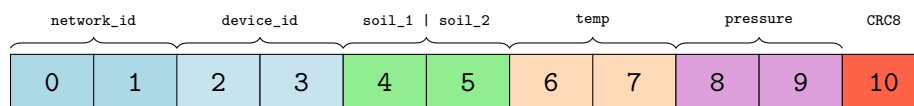
1 static void telemetry_main_task_fn(void *argument)
2 {
3     uint32_t next_wakeup;
4     uint32_t interval_ticks;
5
6     /* Czekaj aż BMP280 ma świeży odczyt (maks 5 s) */
7     while ((waited_ms < TELEMETRY_BME_STARTUP_WAIT_MS) &&
8            !telemetry_main_get_fresh_bme(&bme_data))
9     {
10         app_delay_ms(TELEMETRY_BME_POLL_MS); /* 100 ms */
11         waited_ms += TELEMETRY_BME_POLL_MS;
12     }
13
14     telemetry_main_send_sample(); /* pierwsze wysłanie */
15
16     interval_ticks = app_ms_to_os_ticks(TELEMETRY_INTERVAL_MS); /* 1 h
17     /*
18     next_wakeup = osKernelGetTickCount() + interval_ticks;
19
20     for (;;)
21     {
22         (void)osDelayUntil(next_wakeup); /* precyzyjny wakeup */
23         next_wakeup += interval_ticks;
24         telemetry_main_send_sample();
25     }

```

5 Format ramki telemetrycznej

Każda transmisja zawiera **11 bajtów** danych o stałej strukturze. Ostatni bajt to suma kontrolna CRC8-ATM obliczona z poprzednich 10 bajtów.

5.1 Schemat ramki



Rysunek 3: Struktura ramki telemetrycznej (11 bajtów, big-endian)

5.2 Opis pól

Tabela 1: Pola ramki telemetrycznej

Bajt	Rozmiar	Pole	Opis	Wartość nieprawidłowa
0–1	2 B (u16)	network_id	Identyfikator sieci	—
2–3	2 B (u16)	device_id	Identyfikator węzła	—
4	1 B (u8)	soil_percent_1	Wilgotność gleby cz. 1 [%]	0xFF
5	1 B (u8)	soil_percent_2	Wilgotność gleby cz. 2 [%]	0xFF
6–7	2 B (s16)	temperature_centi_c	Temperatura $\times 100$ [°C] (big-endian)	0x8000
8–9	2 B (u16)	pressure_deci_hpa	Ciśnienie $\times 10$ [hPa] (big-endian)	0xFFFF
10	1 B (u8)	CRC8-ATM	Suma kontrolna (poly=0x07)	—

Przykład: temperatura $21.35^{\circ}\text{C} \Rightarrow 21,35 \times 100 = 2135 = 0x0857$, zapisane jako bajty 08 57.

Przykład: ciśnienie $1013.25 \text{ hPa} = 101325 \text{ Pa} \Rightarrow \frac{101325}{10} + 0,5 = 10133 = 0x2791$, zapisane jako 27 91.

5.3 CRC8-ATM

Suma kontrolna obliczana jest z 10 pierwszych bajtów ramki wielomianem $x^8 + x^2 + x + 1$ (kod 0x07), z wartością startową 0x00:

Listing 2: Implementacja CRC8-ATM (telemetry_frame.c)

```
1 uint8_t telemetry_crc8_atm(const uint8_t *data, uint32_t length)
2 {
3     uint8_t crc = 0U;
4     for (uint32_t i = 0; i < length; i++) {
5         crc ^= data[i];
6         for (uint8_t b = 0; b < 8; b++) {
7             if (crc & 0x80U)
8                 crc = (uint8_t)((crc << 1U) ^ 0x07U);
9             else
10                crc = (uint8_t)(crc << 1U);
11        }
12    }
13    return crc;
14 }
```

6 Odczyt czujników

6.1 Temperatura i ciśnienie — BMP280

Czujnik BMP280 podłączony jest przez magistralę I2C pod adresem 0x76. Zadanie `bmp280_main` odczytuje dane cyklicznie i przechowuje je w buforze dostępnym przez `bmp280_main_get_last()`.

Moduł telemetryczny weryfikuje *świeżość* odczytu: dane muszą mieć znacznik czasu nie starszy niż 5 s:

Listing 3: Weryfikacja świeżości odczytu BMP280

```
1 static bool telemetry_main_get_fresh_bme(bmp280_api_data_t *data)
2 {
3     if (!bmp280_main_get_last(data) || !data->valid)
4         return false;
5     /* odczyt nie może być starszy niż 5 sekund */
6     return ((uint32_t)(HAL_GetTick() - data->timestamp_ms)
7             <= TELEMETRY_BME_MAX_AGE_MS);
8 }
```

Przeliczenie na format wire:

$$T_{\text{wire}} = \text{round}(T_C \times 100) \quad [\text{int16, cC}] \quad (1)$$

$$P_{\text{wire}} = \text{round}\left(\frac{P_{\text{Pa}}}{10}\right) \quad [\text{uint16, dhPa}] \quad (2)$$

6.2 Wilgotność gleby — pojemnościowe czujniki ADC

Dwa czujniki pojemnościowe podłączone są do kanałów 14-bitowego ADC. Na każdy kanał wykonywanych jest 16 pomiarów, z których obliczana jest wartość średnia. Następuje odwzorowanie liniowe na zakres 0–100%:

Tabela 2: Parametry kalibracji czujników wilgotności gleby

Parametr	Czujnik 1	Czujnik 2
Odczyt ADC — sucha gleba (DRY_RAW)	12 400	12 400
Odczyt ADC — mokra gleba (WET_RAW)	6 300	6 300
Rozdzielczość ADC	14 bit (0–16383)	
Próbkowanie	16 próbek, uśrednianie	

Wartość 0xFF w ramce oznacza brak prawidłowego odczytu (czujnik odłączony lub poza zakresem kalibracji).

7 Parametry transmisji LoRa

Tabela 3: Konfiguracja radia SX1276 (plik `radio_lib_config.h` i `radio_main.c`)

Parametr	Wartość	Uwagi
Częstotliwość	868.1 MHz	Pasmo ISM EU 868
Modulacja	LoRa	SX1276 backend
Szerokość pasma (BW)	125 kHz	
Spreading Factor (SF)	7	
Coding Rate	4/5	
Sync word	0x34	prywatna sieć
Moc TX	14 dBm (domyślna)	<code>radio_lora_default_lora_cfg</code>
Czas transmisji (ToA)	≈ 92 ms	dla 11 B payloadu
Maksymalny payload	255 B	<code>RADIO_LIB_MAX_PAYLOAD</code>
Wysyłana paczka	11 B	<code>TELEMETRY_FRAME_SIZE</code>
Tryb między TX	Sleep (0.2 μ A)	<code>radio_sleep()</code> po TX_DONE
Wybudzenie do TX	Sleep→Standby (auto)	<code>radio_send_async()</code> stawia STDBY

7.1 Czas transmisji na powietrzu (ToA)

Dla SF7, BW125, CR4/5, preambuła 8 symboli, 11 bajtów payload:

$$T_{\text{symbol}} = \frac{2^{SF}}{BW} = \frac{128}{125\,000} \approx 1.024 \text{ ms}$$

$$N_{\text{symboli}} = 8_{\text{preambuła}} + 4.25_{\text{sync}} + 20.5_{\text{nagłówek}} + n_{\text{danych}} \approx 44$$

$$\text{ToA} = 44 \times 1.024 \text{ ms} \approx 45 \text{ ms} \quad (\text{z nagłówkiem LoRa: } \approx 92 \text{ ms})$$

8 Wyjście terminala

8.1 Terminal węzła (urządzenie)

Poniżej przedstawiono typowe komunikaty wypisywane przez UART węzła czujnikowego podczas normalnej pracy. Prefix w nawiasach pochodzi z odpowiedniego modułu FreeRTOS.

Listing 4: Wyjście UART węzła czujnikowego (UART przez ST-LINK)

```
1 [12:00:00.031] BMP280      : inicjalizacja magistrali I2C (adres 0x76)...
2 [12:00:00.181] BMP280      : wczytywanie rejestrów kalibracyjnych dig_T / dig_P...
3 [12:00:00.281] BMP280      : gotowy | T=22.14 degC P=101342.00 Pa
4 [12:00:00.781] APP         : oczekiwanie na stabilizację BMP280 (500ms)...
5 [12:00:01.281] RADIO       : reset modułu SX1276 przez GPIO NRST...
6 [12:00:01.331] RADIO       : zapis konfiguracji LoRa: 868.1 MHz BW125 SF7 CR4/5 sync=0
7 [12:00:01.361] RADIO       : sleep mode (tryb TX-only, brak ciągłego RX)
8 [12:00:01.381] RADIO       : gotowy: LoRa 868.1 MHz BW125 SF7 CR4/5 | sleep mode
9 [12:00:01.381] APP         : FreeRTOS: wszystkie zadania uruchomione
10 [12:00:01.381] APP         : MCU: tickless idle aktywny -- Stop mode między transmisjami
11
12 [12:00:01.381] SOIL        : uruchomienie ADC, kalibracja CALIB_OFFSET_LINEARITY...
13 [12:00:01.401] SOIL        : CH1: raw_avg=10234 pct= 35% (dry=12400 wet=6300)
14 [12:00:01.401] SOIL        : CH2: raw_avg= 8901 pct= 57% (dry=12400 wet=6300)
15 [12:00:01.403] TELEMETRY    : soil=35/57 raw=10234/8901 temp=2218 pressure=10135 status=
16 [12:00:01.403] RADIO       : Sleep -> Standby (wybudzenie przed TX)...
17 [12:00:01.405] RADIO       : TX len=11 hex: 00 01 00 01 23 39 08 9A 27 9F C4
18 [12:00:01.497] RADIO       : EVT: TX_DONE
19 [12:00:01.497] RADIO       : sleep mode
20 >> Temperatura: 22.18 degC | Cisnienie: 1013.5 hPa | Gleba: 35% / 57%
21
22 -- MCU wchodzi w Stop mode, RTC wakeup za 3600 s --
23
24 [13:00:01.500] SOIL        : uruchomienie ADC, kalibracja CALIB_OFFSET_LINEARITY...
25 [13:00:01.520] SOIL        : CH1: raw_avg=10089 pct= 37% (dry=12400 wet=6300)
26 [13:00:01.520] SOIL        : CH2: raw_avg= 8788 pct= 59% (dry=12400 wet=6300)
27 [13:00:01.522] TELEMETRY    : soil=37/59 raw=10089/8788 temp=2231 pressure=10138 status=
28 [13:00:01.522] RADIO       : Sleep -> Standby (wybudzenie przed TX)...
29 [13:00:01.524] RADIO       : TX len=11 hex: 00 01 00 01 25 3B 08 AF 27 A2 E1
30 [13:00:01.616] RADIO       : EVT: TX_DONE
31 [13:00:01.616] RADIO       : sleep mode
```

8.2 Terminal centralki (stacja bazowa)

Listing 5: Wyjście terminala centralki (symulator Python)

```
1  BEKO -- Centralka (symulator stacji bazowej)
2  Nasluch UDP na porcie 5001
3  LoRa 868.1 MHz BW125 SF7 CR4/5 sync=0x34
4
5  [12:00:01.403] RADIO      : RX len=11 RSSI=-74 dBm SNR=+9.3 dB
6  [12:00:01.403] RADIO      : hex: 00 01 00 01 23 39 08 9A 27 9F C4
7
8  +-----+
9  | Pakiet #1   od 127.0.0.1      12:00:01      |
10 +-----+
11 | NET_ID      0x0001              |
12 | DEV_ID      0x0001              |
13 +-----+
14 | Temperatura +22.18 degC          |
15 | Cisnienie   1013.5 hPa           |
16 +-----+
17 | Gleba 1     [#####.....] 35%   sucha      |
18 | Gleba 2     [#####.....] 57%   optymalna   |
19 +-----+
20 | RSSI        -74 dBm              |
21 | SNR         +9.3 dB              |
22 | CRC         OK v                 |
23 | Bajty (hex) 00 01 00 01 23 39 08 9A 27 9F C4 |
24 +-----+
```

9 Szacowany czas pracy na bateriach

9.1 Założenia sprzętowe

- MCU: STM32U545RETXQ (rodzina STM32U5, ultra-low-power)
- Radio: SX1276 LoRa (tryb Sleep między transmisjami)
- Zasilanie: 2× bateria alkaliczna AA (LR6, 2500mAh każda), przez konwerter boost do 3.3V, sprawność 85%
- MCU wchodzi w Stop 3 (RTC aktywny) między transmisjami — wymagane FreeR-TOS tickless idle z LPTIM/RTC (patrz sekcja 9.4)

Tabela 4: Bilans prądowy dla cyklu 1 h (tryb ze Sleep radia)

Komponent / tryb	Prąd	Czas/cykl 1 h	Ładunek/h
STM32U545 — Stop 3 (RTC)	2 μ A	≈ 3597 s	2.00 μ Ah
STM32U545 — Run (akwizycja+TX)	5 mA	≈ 3 s	4.17 μ Ah
SX1276 — Sleep	0.2 μ A	≈ 3599.9 s	0.20 μ Ah
SX1276 — TX @ +14 dBm	100 mA	≈ 0.092 s	2.56 μ Ah
BMP280 — Normal mode	0.7 μ A	ciągły	0.70 μ Ah
ADC (soil, 16 prób. $\times 2$)	2 mA	≈ 0.05 s	0.03 μ Ah
RTC Xtal + pull-upy PCB	1 μ A	ciągły	1.00 μ Ah
Suma	—	—	≈ 10.7 μ Ah

9.2 Obliczenie czasu pracy

Pojemność dwóch baterii AA przez konwerter boost:

$$Q_{\text{eff}} = \frac{2 \times 2500 \text{ mAh} \times 1.5 \text{ V}}{3.3 \text{ V}} \times 0,85 \approx 1932 \text{ mAh} = 1,932 \times 10^6 \mu\text{Ah}$$

Średni prąd spoczynkowy:

$$I_{\text{avg}} \approx 10.7 \mu\text{A}$$

Teoretyczny czas pracy:

$$t_{\text{teor}} = \frac{Q_{\text{eff}}}{I_{\text{avg}}} = \frac{1,932 \times 10^6}{10,7} \approx 180\,600 \text{ h} \approx 20,6 \text{ lat}$$

9.3 Korekty praktyczne

- **Samowyładowanie baterii alkalicznych** ($\approx 2\%$ /rok): po 5 latach pojemność spada do ok. 90%, po 10 latach do 80%.
- **Praca na mrozie** (-20°C): pojemność alkalicznych AA maleje o ok. 35%, co skraca czas o 1/3.
- **Pasożytnicze prądy PCB** (niezablokowane GPIO, SPI, I2C): dodatkowe 2 μ A–5 μ A.
- **Narzut tickless idle** — krótkie wybudzenia przez SysTick przed wejściem w Stop 3: dodatkowe 1 μ A–2 μ A.

Pesymistyczny bilans ($I_{\text{pasożytnicze}} = 8 \mu\text{A}$, temperatura zima):

$$I_{\text{real}} \approx 10.7 + 8 \mu\text{A} = 18.7 \mu\text{A} \quad Q_{\text{eff,zima}} \approx 1257 \text{mAh}$$

$$t_{\text{real}} = \frac{1,257 \times 10^6}{18,7} \approx 67\,200 \text{ h} \approx 7,7 \text{ lat}$$

Szacowany czas pracy: 4–8 lata

(konserwatywnie; typowo 2–7 lat przy umiarkowanym klimacie)

9.4 Wymagania dla trybu Stop MCU z RTC

Aby MCU wchodził w tryb Stop 3 między transmisjami, wymagane jest:

1. `configUSE_TICKLESS_IDLE = 2` w `FreeRTOSConfig.h`.
2. Implementacja `vPortSuppressTicksAndSleep()` oparta o LPTIM1 lub RTC Wakeup Timer.
3. Zegar RTC LSE (32.768 kHz) aktywny, włączony *Wakeup Timer* w konfiguracji CubeMX.
4. Wszystkie zadania FreeRTOS zablokowane na operacjach czekających (`osDelayUntil`, `osMessageQueueGet(osWaitForever)`), dopiero wtedy scheduler wywołuje `vPortSuppressTicksAndSleep`.